

Eat Acid / ea-xyz

Protocol, Collection Logic & Downstream Systems Reference

A consolidated planning document for the collection economy, burn mechanics, cross-chain architecture, Potlatch, and Sacred Drip.

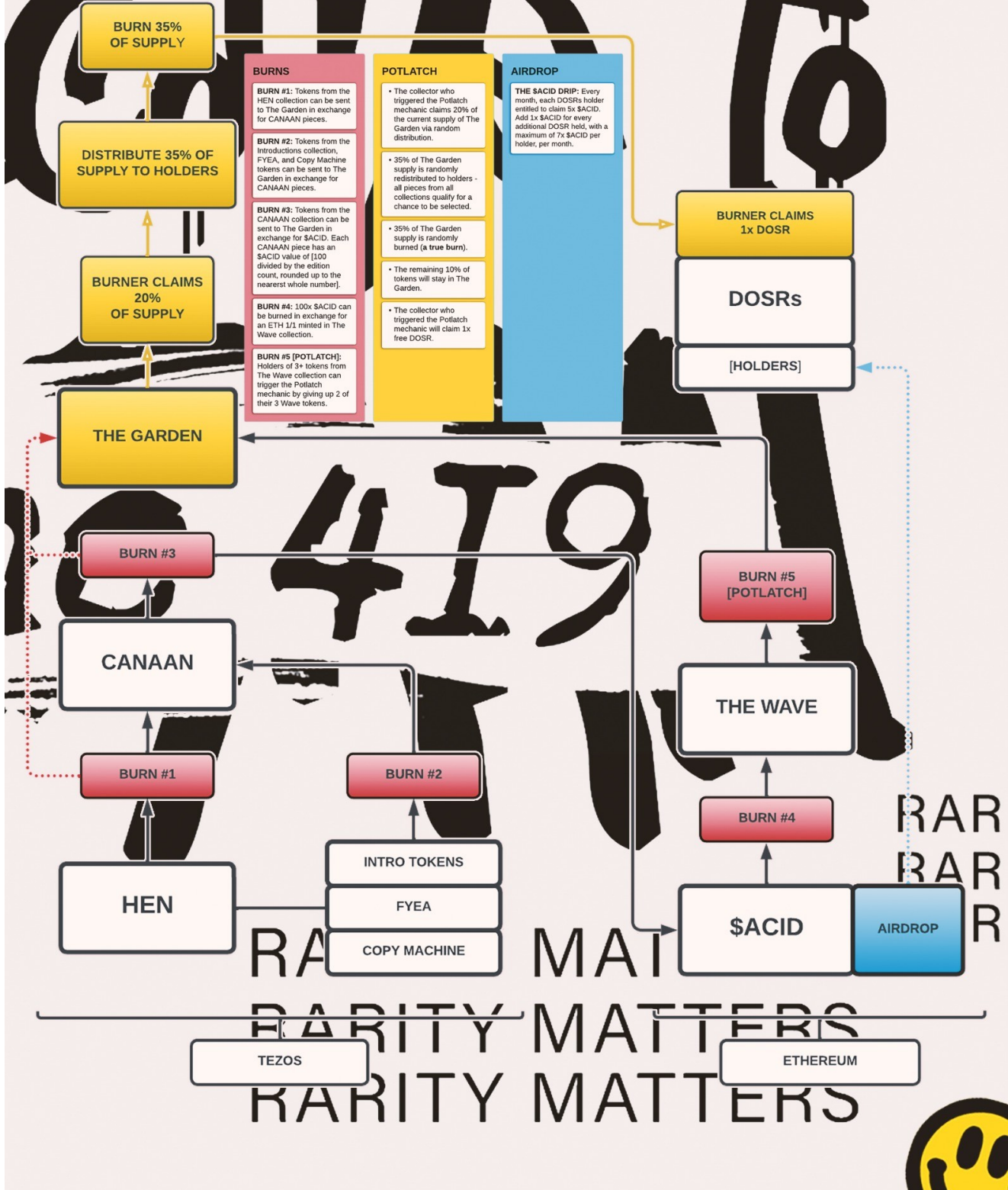
DOCUMENT PURPOSE

This document captures the current shared understanding of the project so future implementation work can begin from a stable reference instead of reconstructing the system from conversation history.

CURRENT — Burns #1–3 and the Tezos-side base layer are the implemented/established foundation referenced by this plan.

PROPOSED — Burn #4, Potlatch, DOSR minting, Sacred Drip, and the cross-chain coordinator are downstream architecture to be designed and implemented.

OPEN DECISION — Several protocol-level details remain intentionally unresolved; they are collected in the final section rather than silently assumed.



Canonical concept flowchart used for this planning pass.

1. Executive Model

ONE-SENTENCE MODEL

Eat Acid is a one-way, progressively deflationary art economy in which historical works are sacrificed to unlock later stages, creating inward pressure on older secondary markets while turning uncertainty, risk, sacrifice, and redistribution into part of the artwork itself.

The system operates on three overlapping layers:

- Economic — finite upstream supply is progressively removed from circulation, so downstream participation creates scarcity and demand pressure backward through the project history.
- Strategic / game-theoretic — collectors are repeatedly asked to trade known value for uncertain outcomes, accept asymmetric risk, or act when others may hesitate.
- Narrative / mythic — the collector performs a journey of departure, sacrifice, transformation, ordeal, dissolution, redistribution, and return rather than merely reading those themes in the artwork.

The Garden

CURRENT — The Garden is the project burn wallet / sink. Assets surrendered through the Tezos burn-and-exchange pipeline are sent into The Garden.

This distinction is important: The Garden is not merely another collection in the lineage. It is the accumulating repository of sacrificed works and, downstream, the inventory pool that Potlatch acts upon.

The one-way ratchet

CORE DIRECTION

Closed historical supply → sacrifice → scarce intermediate work → \$ACID → cross-chain Wave → Potlatch → redistribution / DOSR → recurring \$ACID.

2. Collection & Acquisition Logic

Element	Role in the system	Key constraint / property
HEN	Closed historical source collection.	Closed; cannot replenish. Qualifying HEN-era works are sacrificed to obtain CANAAN.
Intro Tokens / FYEA / Copy Machine	Additional qualifying historical sources for CANAAN.	Qualifying assets are surrendered into The Garden.
CANAAN	Scarce intermediate collection / gateway.	Not directly minted or purchased through the protocol; obtained through historical burns, then itself surrendered for \$ACID.
419	Open / liquid alternate on-ramp to \$ACID.	Could and can be purchased outright; historically less XTZ-efficient per \$ACID than the HEN → CANAAN path.

Element	Role in the system	Key constraint / property
\$ACID	Fungible transformation / bridge asset.	Obtained through CANAAN and 419 pathways; 100 \$ACID is the planned Burn #4 threshold for a Wave.
The Wave	Ethereum 1/1 collection downstream of \$ACID.	Cross-chain redemption from Tezos; later supplies the Potlatch qualification mechanic.
DOSR	Ethereum reward/receipt class created through Potlatch.	Creates recurring Sacred Drip entitlement back on Tezos.
The Garden	Burn wallet and later Potlatch inventory pool.	Receives surrendered assets; Potlatch redistributes/burns portions of its inventory.

Historical pricing intent

CURRENT — The design intent was for the HEN → CANAAN → \$ACID route to reward deeper participation compared with simply purchasing 419.

At the time of design, the remembered XTZ-to-\$ACID economics through 419 were roughly 25–50% worse than proceeding through the historical HEN pipeline. The current market ratio has not been revalidated and should be treated as a historical design reference, not a current price claim.

3. Burn / Exchange Mechanics

Burn	Source	Result	Chain	System meaning
#1	Qualifying HEN works	CANAAN	Tezos	Historical work is surrendered into The Garden to enter the CANAAN stage.
#2	Intro Tokens / FYEA / Copy Machine	CANAAN	Tezos	Additional historical routes consolidate into CANAAN.
#3	CANAAN	\$ACID	Tezos	CANAAN is surrendered and converted into fungible \$ACID value.
#4	100 \$ACID	1 Wave	Tezos → Ethereum	Cross-chain threshold: Tezos-side sacrifice creates an Ethereum Wave entitlement.
#5 / Potlatch	Holder has 3 Waves; sacrifices 2	Garden event + DOSR	Ethereum → Tezos (+ ETH DOSR)	A user gives up 2 of 3 Waves to trigger

Burn	Source	Result	Chain	System meaning
				communal redistribution/burn and receive a DOSR.

CANAAN → \$ACID valuation

The flowchart defines each CANAAN piece's \$ACID value as: **100 ÷ edition count, rounded up to the nearest whole \$ACID.**

- Edition of 10 → 10 \$ACID.
- Edition of 33 → 4 \$ACID.

4. Cross-Chain Architecture

SCOPE

There are three actual cross-chain references in the downstream system. Wallet pairing is shared identity infrastructure, not a fourth state reference.

#	Cross-chain reference	Direction	Destination needs to know
1	\$ACID burn → Wave claim	Tezos → Ethereum	A valid Tezos Burn #4 occurred for the correct amount and the specified paired Ethereum recipient.
2	Potlatch trigger → Garden execution	Ethereum → Tezos	A valid Wave holder satisfied and executed the Potlatch surrender rules.
3	DOSR ownership → Sacred Drip entitlement	Ethereum → Tezos	At the defined epoch/claim point, the paired Ethereum wallet owns N DOSRs.

Dual-wallet pairing

PROPOSED — The app connects a Tezos wallet and an Ethereum wallet simultaneously and asks both wallets to sign the pairing relationship.

- Canonical relationship: tz1... ↔ 0x...
- Both sides should sign so the pair is cryptographically authorized by both wallets.
- Cross-chain actions use the paired identity rather than guessing ownership across chains.
- For irreversible cross-chain burns, embed the destination wallet in the source-chain event/record so the intended recipient remains reconstructable from chain history.

Coordinator vs. trust-minimized bridge

The practical distinction is not whether a middleman exists. Both models require something to transport information. The distinction is whether that middleman is trusted as an authority or merely acts as a messenger.

Model	What destination contract verifies	If middleman lies	Fit for EA
Custom coordinator / purpose-built oracle	A signed assertion from the approved Eat Acid coordinator.	The destination cannot independently prove the source fact; signer security is therefore critical.	Preferred: simpler, narrow, auditable, and aligned with the priority of security/correctness over permanent trustless provenance.
Trust-minimized state-proof bridge	Proof ultimately anchored to source-chain state/consensus.	A bad relayer cannot forge a valid source-state proof.	Possible but materially more complex; not currently justified by project priorities.

DESIGN PRIORITY

Optimize for security and correctness. Eat Acid does not currently need to solve generalized trustless Ethereum ↔ Tezos interoperability or permanent cross-chain provenance.

Coordinator security model

- Only recognize allowlisted source contracts and exact event types.
- Wait for an explicit confirmation/finality policy before authorizing destination actions.
- Build canonical, deterministic messages rather than interpreting ambiguous API data.
- Use unique event IDs/nonces and destination-side replay protection.
- Make processing idempotent: retries must never duplicate mint/distribution actions.
- Protect signing keys aggressively; support controlled key rotation and emergency pause.
- Preserve an audit trail linking source transaction, attestation, destination transaction, and final status.

5. Full System Inventory

The complete scope breaks into nine functional systems plus two shared operational layers.

1. Tezos Burn / Exchange Base Layer

CURRENT — Burns #1–3; qualifying collection resolution; CANAAN/\$ACID exchange logic; surrender into The Garden; consumed-asset protection.

2. Dual-Wallet Identity & Pairing

PROPOSED — Connect Tezos + Ethereum; dual signatures; persistent/revocable pair state; shared identity for every cross-chain mechanic.

3. Cross-Chain Coordinator / Oracle

PROPOSED — Watch both chains; confirm source events; validate exact parameters; construct canonical messages; sign attestations; track replay IDs; retry safely; audit everything.

4. \$ACID → Wave Redemption

PROPOSED — Burn #4. Tezos burn of 100 \$ACID tied to paired ETH recipient; coordinator confirms; Ethereum Wave contract verifies attestation and mints/assigns one Wave exactly once.

5. Wave / Potlatch Trigger

PROPOSED — Ethereum-side qualification. Confirm 3 Waves; sacrifice 2; emit unique Potlatch event; establish DOSR recipient; prevent reuse of surrendered Waves.

6. Garden Inventory & Distribution Engine

PROPOSED — Know current Garden inventory; compute Potlatch allocations; execute 20% claimant / 35% holders / 35% burn / 10% remain; define rounding and partial-failure behavior.

7. Holder Eligibility & Snapshot Engine

PROPOSED — Define qualifying collections and holders at a canonical block/level; exclusions; marketplace escrow handling; duplicate wallets; weighting rules.

8. Randomness & Selection System

PROPOSED — Generate auditable, manipulation-resistant randomness for holder distribution and Garden burn selection; preserve enough data to reproduce the outcome.

9. DOSR + Sacred Drip

PROPOSED — Ethereum DOSR mint/ownership; recurring ETH→Tezos ownership reference; calculate monthly \$ACID entitlement; preferably expose a claim-based Tezos payout.

Shared operational layer A — Protocol ledger / audit history

- Canonical event ID and action type.
- Source chain, source contract, transaction, and block/level.
- Source wallet and paired destination wallet.
- Assets / amount / token IDs involved.
- Coordinator attestation and signer version.
- Destination transaction and final status.
- For Potlatch: snapshot identifiers, random seed/proof, selected Garden assets, recipients, and burns.

Shared operational layer B — Admin / emergency controls

- Pause new cross-chain claims.
- Pause Potlatch execution.
- Rotate a compromised coordinator key.
- Inspect and retry stuck but safe events.
- Disable a broken source/contract integration.
- Do not permit arbitrary rewriting of already finalized protocol history.

6. Burn #4 — \$ACID → The Wave

TARGET FLOW

Pair wallets → burn 100 \$ACID on Tezos → record paired ETH recipient + unique burn ID → wait for confirmation/finality → coordinator signs entitlement → Ethereum verifies → Wave is minted/assigned → burn ID is permanently marked consumed.

Canonical source event should contain

- Burn / redemption ID.
- Tezos source wallet.
- Ethereum destination wallet.
- \$ACID amount.
- Source contract / action type.
- Enough event/state data to unambiguously determine that this is a Wave redemption rather than a generic token transfer.

Destination invariants

- Approved attestation only.
- Correct chain/domain and contract context.
- Exactly one Wave entitlement per valid 100-\$ACID redemption unit, per final rule.
- A burn/redemption ID can never be consumed twice.
- Wave can only be delivered to the destination committed by the source-side action, unless an explicit recovery rule exists.

7. Potlatch

PROPOSED — A Wave holder with three qualifying Waves may trigger Potlatch by surrendering two of the three. The event reaches back across chains and acts on The Garden.

Potlatch distribution model

Allocation	Action	Interpretation
20%	Claimed by the triggering collector	Direct reward for taking the irreversible action.
35%	Randomly distributed to qualifying holders	The individual action creates a communal redistribution event.
35%	Randomly burned	Further irreversible deflation of the Garden inventory.
10%	Remains in The Garden	Persistent residue / continuation of the Garden state.

Execution pipeline

1. User proves/uses the required Wave position on Ethereum.

2. Two Waves are irrevocably surrendered under the final contract rule.
3. Ethereum emits a unique Potlatch trigger event.
4. Coordinator waits for finality and validates the trigger.
5. Holder and Garden snapshots are fixed at protocol-defined reference points.
6. Randomness is fixed and the complete allocation/burn plan is computed.
7. Tezos Garden distribution/burn actions execute safely and idempotently.
8. DOSR is minted/assigned on Ethereum according to the chosen completion rule.
9. Audit ledger records the full event so the result can be independently reconstructed.

Why Potlatch is the hardest subsystem

The cross-chain reference itself is narrow: “a valid Potlatch trigger happened on Ethereum.” The complexity comes from defining and safely executing holder eligibility, Garden inventory accounting, randomness, percentage rounding, multi-asset distribution, burn selection, and failure recovery.

8. DOSR & Sacred Drip

CURRENT — DOSRs are intended to live on Ethereum.

PROPOSED — DOSR ownership creates recurring \$ACID entitlement on Tezos through the shared cross-chain coordinator.

Monthly entitlement

DOSRs held	Monthly \$ACID
1	5
2	6
3 or more	7 (cap)

This reflects the stated rule: 5 \$ACID for the first DOSR, +1 \$ACID for each additional DOSR, capped at 7 \$ACID per holder per month.

Preferred payout model

CLAIM-BASED SACRED DRIP

The system automatically establishes monthly entitlement, but the holder claims the \$ACID when desired rather than the coordinator blindly pushing transfers. This reduces failed transactions and operating cost while keeping entitlement deterministic.

9. Shared Cross-Chain Message Pattern

All three cross-chain mechanics should use one common envelope even though their payloads differ. A suggested conceptual schema:

```
sourceChain
sourceContract
sourceBlockOrLevel
```

sourceTransaction
 eventId
 actionType
 sourceWallet
 destinationWallet
 amountOrTokenIds
 epochOrSnapshotId (when applicable)
 nonce / replay domain

Destination-side execution pattern: verify attestation → validate domain/context → check event unused → execute invariant-constrained action → mark event consumed → emit destination event.

10. Recommended Implementation Roadmap

Phase	Build	Why this order
A — Cross-chain foundation	Dual-wallet pairing; shared event/message schema; coordinator; signer security; replay ledger; admin pause/recovery.	Everything downstream reuses this plumbing.
B — Burn #4	End-to-end \$ACID → Wave.	Smallest complete cross-chain mechanic and best proving ground for the coordinator.
C — Potlatch rules/infrastructure	Wave trigger contract; Garden inventory model; holder eligibility/snapshots; randomness design.	Define irreversible rules before building the executor.
D — Potlatch execution	Tezos distribution/burn engine; cross-chain trigger handling; DOSR mint/completion semantics.	Largest and highest-risk subsystem.
E — Sacred Drip	DOSR epochs/snapshots; recurring entitlement; Tezos claim path.	Build on already-proven pairing/coordinator/replay infrastructure.

11. Protocol Decisions Still to Lock

RULE BEFORE CODE

The items below are protocol semantics, not implementation trivia. They should be deliberately decided before the associated irreversible contract logic is finalized.

Decision	Question to resolve
Wallet pairing lifecycle	Can a pair be changed? Is there a cooling-off period? What happens to pending claims when one side is repaired?

Decision	Question to resolve
Burn #4 unit semantics	Is 100 \$ACID always exactly one Wave? Can a single transaction redeem multiples of 100? What happens to non-multiple amounts?
Wave surrender semantics	Are the two Waves burned, transferred into a designated sink, or otherwise permanently disabled?
Potlatch trigger completion	When is the trigger considered irrevocably complete: after ETH surrender, after Tezos execution, or only after both + DOSR mint?
Garden inventory definition	Which assets inside the Garden count toward the Potlatch percentages? Are any classes reserved/excluded?
Percentage rounding	How are 20/35/35/10 allocations rounded for indivisible NFTs, and which bucket receives remainder assets?
Holder eligibility	Exactly which collections qualify? One chance per wallet, per token, or weighted? How are escrow/marketplace contracts handled?
Snapshot timing	What block/level fixes holder eligibility and Garden inventory relative to the Potlatch trigger?
Randomness source	What manipulation-resistant, auditable method fixes recipient and burn selection?
DOSR mint timing	Mint immediately on Ethereum trigger, or only after Garden execution is confirmed?
Sacred Drip epoch	Fixed monthly snapshot vs. claim-time ownership; UTC/calendar boundary; treatment of DOSR transfers within an epoch.
Unclaimed Sacred Drip	Does entitlement expire, roll over, accumulate forever, or return to treasury?
Coordinator confirmation policy	How much Ethereum/Tezos finality is required before each irreversible destination action?
Admin recovery boundaries	Exactly what can be paused/retried/recovered without creating an admin power that can rewrite protocol outcomes?

12. Reference Architecture — At a Glance

TEZOS BASE LAYER

Historical collections → Burns #1/#2 → CANAAN → Burn #3 → \$ACID → Burn #4 event

CROSS-CHAIN FOUNDATION

Dual-wallet pairing + coordinator/oracle + canonical attestations + replay protection + audit ledger

ETHEREUM

Burn #4 entitlement → The Wave → Potlatch trigger / 2-of-3 Wave surrender → DOSR

TEZOS DOWNSTREAM

Potlatch trigger → Garden snapshot/distribution/burn → 20% claimant / 35% holders / 35% burn / 10% remain

RECURRING LOOP

Ethereum DOSR ownership → coordinator → Tezos Sacred Drip entitlement → \$ACID claim

Bottom line

The remaining project is not “three different bridges.” It is one reusable cross-chain identity/attestation foundation supporting three narrow references, plus two substantial domain systems: Potlatch/Garden execution and recurring DOSR → \$ACID entitlement. The technical priority is deterministic correctness, replay safety, signer security, transparent event history, and controlled failure recovery—not generalized trustless interoperability.